

# DECISION TREES

## Análise de Dados em Informática Licenciatura em Engenharia Informática ISEP/IPP

Ana Maria Madureira  
2024/2025

### Fontes:

- Tom Mitchell, Machine Learning, McGraw-Hill, 1997.
- Catarina Silva e Bernardete Ribeiro, Aprendizagem Computacional em Engenharia, Imprensa da Universidade de Coimbra, 2018
- Sebastian Raschka, STAT479 FS18. L01: Intro to Machine Learning, Fall 2018

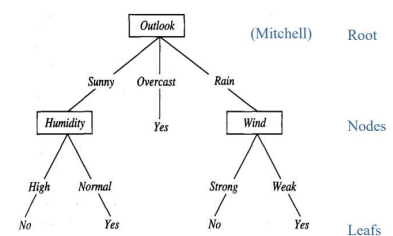
52

52

# DECISION TREES - REPRESENTATION OF CONCEPTS

- A decision tree (DT) consists of a set of decision nodes, connected by branches, extending downward from the root node until terminating in leaf nodes. Beginning at the root node, which by convention is placed at the top of the decision tree diagram, variables are tested at the decision nodes, with each possible outcome resulting in a branch. Each branch then leads either to another decision node or to a terminating leaf node.
- Learning in DT is a method of approximating discrete-value target function represented in a decision tree.
- The learned trees may also be represented by a set of if-then rules to improve human readability and interpretability.
- These learning methods are among the most used and have been applied to a large number of fields (since medical diagnosis to credit risk management in banking)

Concept: "good day to play tennis"



Binary decision function: Yes / No

53

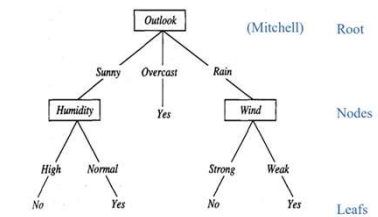
53

# DECISION TREES



- The Decision Tree classifies instances of the problem by descending the tree from the root to some leaf that gives the classification of the instance.
- Each node executes a test over some attribute of the instance, and each descending branch from that node corresponds to one of the possible values of that attribute.
- The classification process of an instance starts at the root, testing the attribute specified by this node, descending the branch of the tree corresponding to the value of the attribute in the example, and going to the extreme node of this branch, where the test of its attribute is made, and the process continues until a leaf of the tree is reached.

Concept: "good day to play tennis"

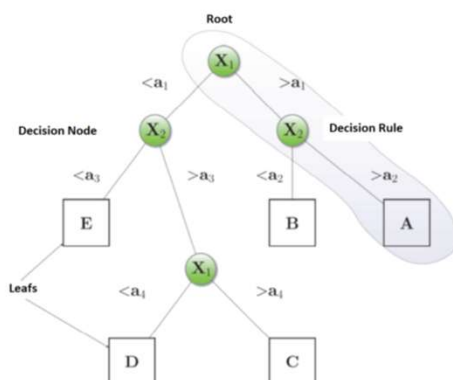


Binary decision function: Yes / No

54

54

# DECISION TREES

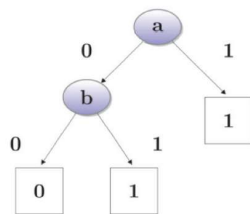


- For the DT representation, we use decision numbers that contain a test related to a certain attribute. Each descending branch corresponds to a possible value (or range of values) for that attribute. Each leaf is associated with a class, and each path in the tree (from the root to the leaf) corresponds to a classification rule.
- In the space defined by the attributes, each leaf corresponds to a region, being represented by a rectangle.
- The intersection of the rectangles is empty, and the union is the complete space
- A DT represents the disjunction of the set of restrictions in the values of the attributes
- Each path in the branch in the tree is a set of conditions, while the set of branches in the tree is disjoint.
- Any logical function can be represented by a decision tree. The result, is found in the right terminal rectangle.

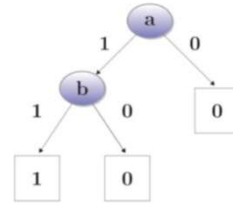
55

55

## DECISION TREES



Representation of the logical function OR with a DT



Representation of the logical function AND with a DT

56

56

## DECISION TREE CONSTRUCTION ALGORITHM

### Decision Tree construction algorithm

#### Input: training examples

1. For each attribute
  - 1.1. Calculate the information gain
  - 1.2. Define the attribute with the highest information gain
2. Create a decision node that is divided into branches based on the values of this attribute
3. If the examples are not all separated by class in each branch, re-apply step 1 and step 2 to each node that is still to be divided

#### Output: Decision tree

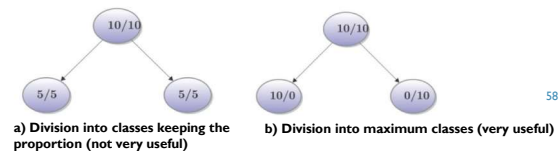
- The process of building a decision tree initially goes through the choosing an attribute that will be the root of the tree. In the example of playing golf of the figure, the chosen one was the weather. The choice of the attribute for the root of the tree is very important.
- It is usually based on information gain. Once chosen this attribute, the "tree" is extended by adding a branch for each value (or range of values) of the attribute, following the examples for the leaves (taking into account the attribute value).
- Then, it is evaluated if the tree is completely built, that is, if all the examples of each branch are of the same class. If not, the process is re-applied to all new nodes in the tree.

57

57

## DECISION TREE CRITERIA FOR CHOOSING AN ATTRIBUTE

- How to measure the ability of a given attribute to discriminate classes?
- There are many measures. The choice of an attribute usually involves the use of heuristics that look a step ahead and normally do not reconsider the options already taken. These heuristics agree on two aspects:
  - A division that maintains the proportions of classes in all partitions that becomes useless, Figure a);
  - A division where in each partition all examples are of the same class has maximum utility, Figure b).
- Partition measures can be characterized in two ways:
  - By the measure of the difference given by the proportions of the classes between the current and the descendants. It has the advantage of enhancing the purity of the partitions.
  - By the measure of the difference given by a function based on the proportions of the classes between the descendants. In this case, the disparity between the partitions is valued, and being a measure of independence, it is also a measure of the degree of association between the attributes and the class.



58

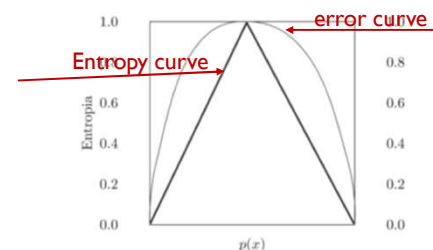
58

## ENTROPY

- Entropy is a measure of the randomness of a variable.
- The entropy of a nominal variable  $X$  that can take  $i$  values can be calculated using the following equation:

$$Entropy(X) = - \sum_i p_i \times \log_2 p_i$$

- Entropy has a maximum ( $\log_2 p_i$ ) if  $p_i = p_j$  if for any different  $i$  of  $j$  and  $Entropy(X) = 0$  if and only if there is an  $i$  that  $p_i = 1$ . It is assumed that  $0 \times \log_2 0 = 0$



59

59

## GAIN INFORMATION

- In the context of DT, entropy is used to estimate the German torment of the variable to predict: the class. The question that arises is to know for a given set of examples, which attribute to choose for test?
- In the context of DT, entropy is used to estimate the randomness of the variable to be predicted: the class.
- The question that arises is: **for a given set of examples, which attribute to choose for testing?**
- We know that the values of an attribute define partitions from the set of examples. On the other hand, the information gain measures the reduction in entropy caused by the partition of the examples according to the attribute values, as shown in the formula:

$$\text{Gain}(\text{Examples}, \text{Attributes}) = \text{Entropy}(\text{examples}) - \sum \frac{\#\text{Examples}_v}{\#\text{Examples}} \times \text{Entropy}(\text{examples})$$

The construction of a DT is guided by the objective of decreasing entropy, that is, the randomness - prediction difficulty - of the objective variable. 60

60

## STOPPING CRITERIA

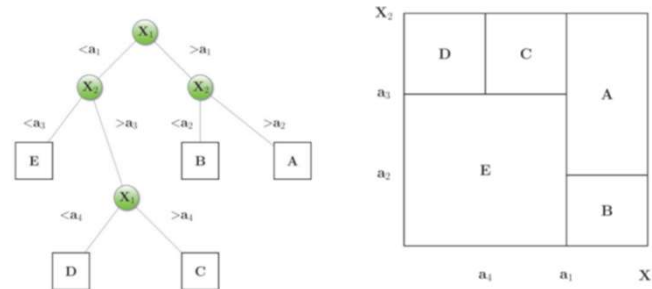
The question that now arises is when should we stop dividing the examples. We can see the following four possibilities:

1. All examples belong to the same class
2. All examples have the same attribute values, but different classes
3. The number of examples is below a certain limit.
4. The merit of all possible partition tests of the examples is very below

61

61

## DECISION TREES



- The DT effect a partition of the space of the attributes as shown in Figure above.
- Let us see how partition A is obtained in the attribute space.
- If the variable  $X_1 > a_1$  and  $X_2 > a_2$ , that is, if the path in the tree is on its right side, from the point to the root, effectively the partition to that found in A.
- we find this partition if we assign these values to the attributes. The reader is invited to check the other partitions from B to E, assigning the possible values of the attributes that configure the branches of the tree on the left side.

62

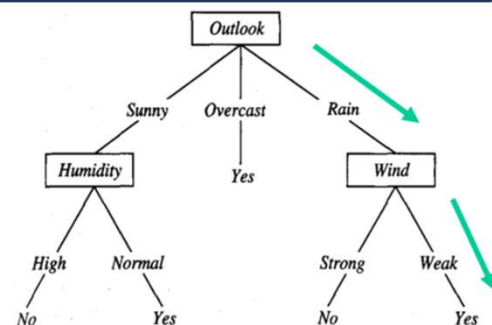
62

## DECISION TREES



Concept: "good day to play tennis"

- Given the instance  $\langle \text{Outlook}=\text{Sunny}, \text{Wind}=\text{Weak} \rangle$
- In this case, analyzing the tree, it is possible to classify (decide) each situation, depending on the weather conditions (e.g., sunny, but it is not wind it means that the decision could be go to play).



63

63

# DECISION TREES

- Decision trees seek to create a set of leaf nodes that are as "pure" as possible, where each of the records in a leaf node has the same classification. The decision tree may provide classification assignments with the highest measure of confidence available.
- We shall examine two of the many methods for measuring leaf node purity, which lead to the two leading algorithms for constructing decision trees:
  - CART algorithm
    - produces DT that are strictly binary, containing exactly two branches for each decision node.
    - recursively partitions the records in the training data set into subsets of records with similar values for the target attribute.
    - grows the tree by conducting for each decision node, an exhaustive search of all available variables and all possible splitting values, selecting the optimal split according to the Gini Index
  - C5.0 algorithm
    - is the J. Ross Quinlan's extension C4.5 algorithm for generating DT
    - is not restricted to binary splits
    - uses the concept of information gain or entropy reduction to select the optimal split. Suppose that we have a variable X whose k possible values have probabilities  $p_1, p_2, \dots, p_k$ . The smallest number of bits, on average per symbol, needed to transmit a stream of symbols representing the values of X observed is called the entropy of X, defined as

$$H(X) = -\sum_i p_i \log_2(p_i)$$

64

J. Ross Quinlan, C4.5: Programs for Machine Learning, Morgan Kaufmann, San Francisco, CA, 1992

64

## DECISION TREES – CLASSIFICATION

```
[ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

#load dataset
db_df = pd.read_csv('diabetes_clean.csv')
print(db_df.shape)
db_df.head()

#we use the loc operator to subset the dataframe, then append .to_numpy() to convert the output into a numpy array in one line
subset_X = db_df.loc[:, ['PhysicalActivity', 'GeneticRisk', 'BloodPressure']].to_numpy()
#considering the features - X - which will be physical activity, genetic risk score and blood pressure.
#showing the first 5 lines
subset_X[0:5]
# y is the outcome, whether or not the person has Diabetes.
y = db_df['Diabetes'].to_numpy()
y[0:5]
```

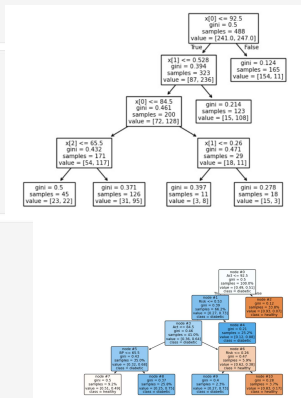
```
[39]: #defining and fitting the tree
from sklearn.tree import DecisionTreeClassifier
tree_model = DecisionTreeClassifier(max_leaf_nodes=6)

#fit the model
tree_model.fit(X_train, y_train)

#visualize the tree
from sklearn.tree import plot_tree
plot_tree(tree_model)
plt.show()
```

```
# define the canvas size so we get a bigger plot
plt.figure(figsize=(18,6))

plot_tree(tree_model,
          #the names of the input features, in the order of the columns in X_train
          feature_names=['Act', 'Risk', 'BP'],
          #show percentage of samples instead of numbers
          proportion=True,
          #color nodes by purity
          filled=True,
          #the names of the classes
          class_names=['healthy', 'diabetic'],
          #show many digits after the dot to show
          precision=2,
          #acute rounded nodes
          rounded=True,
          node_ids=True)
plt.show()
```



Evaluating performance of classifier models

In order to evaluate model performance, we need to apply our trained decision tree to our test data and see what label

```
y_pred = tree_model.predict(subset_X)

print('predicted:')
print(y_pred[10])
print(y_pred.shape, end = '\n\n')

print('true outcome:')
print(y[10])
print(y.shape)

predicted:
[0 0 0 1 1 1 1 1]
(488,)

true outcome:
[0 0 1 0 1 1 1 1]
(488,)

from sklearn.metrics import confusion_matrix
#from sklearn.metrics import classification_report, confusion_matrix
#it's important that we tell the function which are our predictions and which are the true outcomes
pd.DataFrame(confusion_matrix(y_true = y, y_pred = y_pred,
                              indices='Actual_0','Actual_1',
                              columns=['Pred_0','Pred_1'])
```

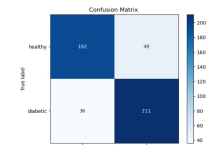
|          | Pred_0 | Pred_1 |
|----------|--------|--------|
| Actual_0 | 192    | 49     |
| Actual_1 | 36     | 211    |

```
print("_____Confusion Matrix_____")
cm = confusion_matrix(y, y_pred)
print(cm)
print("_____Classification Report_____")
print(classification_report(y, y_pred))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['healthy', 'diabetic'])
disp.plot(supplt.cm.Blues)
plt.title("Confusion Matrix")
plt.show()
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.84      | 0.88   | 0.82     | 241     |
| 1            | 0.81      | 0.80   | 0.83     | 247     |
| accuracy     | 0.83 488  |        |          |         |
| macro avg    | 0.83      | 0.83   | 0.83     | 488     |
| weighted avg | 0.83      | 0.83   | 0.83     | 488     |

```
from sklearn.metrics import precision_score, recall_score
round(precision_score(y_true = y, y_pred = y_pred, pos_label = 1),2)
round(recall_score(y_true = y, y_pred = y_pred, pos_label = 1),2)
```

0.85



65

## DECISION TREES REGRESSION

```
import pandas as pd #for data manipulation.
import numpy as np #for scientific computing.
import sklearn #for machine learning
import seaborn as sns #visualization package
import matplotlib.pyplot as plt #plotting & visualization.
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score, mean_absolute_error, root_mean_squared_error, mean_squared_error
from sklearn.preprocessing import StandardScaler
%matplotlib inline
```

```
# Load the Diabetes dataset
diabetes = load_diabetes()
#print(diabetes.DESCR)
```

```
X = diabetes.data
y = diabetes.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

```
#Defining and fitting the tree
from sklearn import tree
```

```
dtree = tree.DecisionTreeRegressor(min_samples_split=20)
dtree.fit(X_train, y_train)
```

```
# Initializing the Decision Tree Regression model
#model = DecisionTreeRegressor(random_state = 0)
# Fitting the Decision Tree Regression model to the data
#dtree.fit(x_train, y_train)
```

```
# Predicting the target values of the test set
y_pred = dtree.predict(X_test)
# RMSE (Root Mean Square Error)
rmse = float(format(np.sqrt(mean_squared_error(y_test, y_pred)), '.3f'))
print("\nRMSE: ", rmse)
```

```
#Visualize the tree
from sklearn.tree import plot_tree
plot_tree(dtree)
plt.show()
```

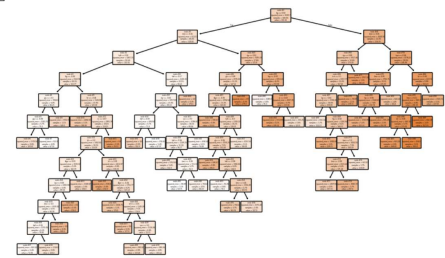
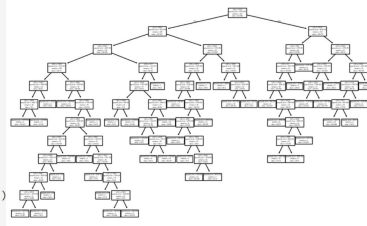
```
#define the canvas size so we get a bigger plot
plt.figure(figsize=(10,6))
```

```
plot_tree(dtree,
           #the names of the input features, in the order of the columns in X_train
           feature_names=['Age', 'sex', 'bmi', 'bp', 'tc', 'ldl', 'hdl', 'tc', 'ltg', 'glu'],
           #show percentage of samples instead of numbers
           proportion=True,
           #color nodes by purity
           filled=True,
           #the names of the classes
           #class_names=['healthy', 'diabetic'],
           #show many digits after the dot to show
           precision=2,
           #acute rounded nodes
           rounded=True,
           node_ids=True)

plt.show()

from sklearn.tree import export_graphviz
# export the decision tree model to a tree_structure.dot file
# paste the contents of the file to webgraphviz.com
export_graphviz(dtree, out_file='tree_structure.dot',
                feature_names=['Age', 'sex', 'bmi', 'bp', 'tc', 'ldl', 'hdl', 'tc', 'ltg', 'glu'])
```

RMSE: 71.235



66

## ATIVIDADE:

- Se estiver a desenvolver um algoritmo para prever se irá chover amanhã, como classificaria o problema: regressão ou classificação? Justifique.
- Imagine que acabou de desenvolver um modelo usando um algoritmo de aprendizagem supervisionada. Após o teste, o modelo apresenta um ótimo desempenho nos dados de treino, mas não é capaz de generalizar em novas instâncias. Este é um caso de overfitting, underfitting ou nenhum dos dois? Justifique.
- As redes neurais representam uma tentativa de imitar o quê? Qual é o principal benefício das redes neurais para modelação?

67

67